



Instituto Politécnico Nacional

Unidad Profesional Interdisciplinaria de Ingeniería y Ciencias Sociales y Administrativas UPIICSA

Ingeniería en informática

Practicas del tercer parcial

Edwin Mendoza Herrera NL:22

Roque Ortiz Iván NL: 36

Yventz Entzana Garduño

Estructura de datos

Repositorio de GitHub:

<https://github.com/saffronprism/Practicas-Estructura-de-Datos.git>



Estructuras de datos con archivos

Ejercicio 1 y 3

Al realizar este ejercicio fue el primero contacto que tuvimos al momento de manejar archivos, teniendo que realizar un ejercicio de ADT y uno de Struct. Al realizarlo tuvimos que ver la implementación para hacer que la pila dinámica pueda guardar su contenido en un archivo txt además de poder abrirlo y cargar la pila en el programa, por lo que optamos por tener en el txt un formato de la posición y del contenido, para que fuera más fácil de entender.

En la pila dinámica con punteros lo que hicimos fue implementar dos métodos con el fin de poder guardar el archivo y abrirlo y cargarlo, usando las funciones de la misma pila para ir eliminando los elementos actuales y poder ir añadiendo los elementos en orden con el fin de que quedaran bien.

Para el CSV fue un poco diferente, ya que al ser un archivo con valores separados en comas, lo que hicimos fue ir poniendo la posición de cada elemento y junto, separado por comas, el valor, para hacerlo de más fácil lectura. Hicimos que cada columna fuera para cada elemento e implementamos la lectura de los elementos de modo que registre el valor.

ERRORES

Al realizar el programa de txt al inicio no leía el archivo, pero esto se debía a que el nombre del archivo está mal, estaba como .txt.txt, lo cual al inicio no se veía pero al detectarlo.

Ejercicio 2 y 4

A partir de este ejercicio, vamos a usar los diferentes proyectos que hemos estado usando, en este caso decidimos agarrar un ADT, en este caso una cola dinámica con la regla estricta FIFO. Al igual que en segundo parcial, se implementó plantillas para hacer mucho menos tediosa la programación de objetos muy repetitiva, esto nos permite manejar 3 tipos de datos primitivos de manera simultánea, asimismo, esto permitió mantener intactos los principios SOLID.

Lo nuevo de estas prácticas es que, tenemos más de un main, uno de ellos es para la escritura y creación de un archivo con datos, el otro main agarra este archivo “muerto”

(tomando en cuenta que tenga el nombre que e pusimos) y lo vuelve a reconstruir, aquí entra la práctica de segundo parcial, en este podemos leer el archivo de texto en crudo, o modificarlo conforme a nuestro ADT elegido, Nuestro ejercicio 2 consistió en hacerlo en .XML, y el numero 4 consistió en hacerlo en .JSON, aun así, el procedimiento y archivos encabezados son casi idénticos entre sí, con unas cuantas diferencias:

Para el XML, lo que usamos son las etiquetas delimitadoras, así como es uso de punteros e índices, al ser un modelo "Case sensitive" una sola mayúscula o una letra faltante podría causar un error de búsqueda y que el programa aparente que esa etiqueta no existe, lo más difícil tanto en XML como en JSON fue coordinar los saltos de índices en cuando los necesitábamos.

Errores

Durante la creación de ambos proyectos, me topé con un bug de memoria basura. El programa original imprimía los tres arreglos (edades, grupos y calificaciones) de golpe y generaba que los datos que no se habían desplegado pareciera que, si lo hicieron, esto generaba "datos fantasmas" con números gigantescos y aleatorios. Solo se tuvo que poner if/else if para poder atacar este problema.

De la misma manera, también tuve problemas con delimitar el que tenía que leer los archivos, ya que estos podían arrojarme etiquetas o identificadores que no existen y el stoi/stof me arrojaba el famoso segmentation fault, lo que tuve que hacer fue calcular es desplazamiento de forma pura y dura, esto me resulto mucho mas fácil.

Grafos y dígrafos

En estos ejercicios empezamos a modelar redes utilizando la teoría de grafos, aplicando fuertemente el paradigma orientado a objetos (POO). Para evitar crear un "código espagueti", dividimos el problema en módulos especializados usando el principio de Responsabilidad Única (SOLID): creé una clase para la captura de datos, otra clase para gestionar el grafo en la memoria RAM, y por último, el motor de archivos que ya venía manejando. De esta forma, el motor de guardado ni siquiera necesita saber qué es un grafo, solo recibe el texto JSON y lo guarda, aplicando el principio Open/Closed.

En esta práctica se hicieron dos variantes. Primero, el **Digrafo** (grafo dirigido), donde el flujo de datos es de un solo sentido (como una calle de un solo sentido). El usuario

define un origen y un destino, y la conexión es estrictamente lineal. Después, implementamos el **Grafo No Dirigido**, el cual simula calles de doble sentido. Para hacer esto y no obligar al usuario a escribir el viaje de ida y luego el de regreso manualmente, apliqué un truco en la lógica del método agregarRuta: el programa recibe la conexión de A hacia B, e inmediatamente y de forma invisible, inyecta la conexión inversa de B hacia A dentro del vector.

Errores

Durante las pruebas de la consola me topé con un error de bucle infinito. Que consta en que, si al momento de pedir un dato numérico (como el tiempo o el costo de una ruta) introducía por accidente una letra o un símbolo, el flujo del cin se rompía y la consola empezaba a ciclarse. Tuve que utilizar `cin.clear()` y `cin.ignore(10000, '\n')` para estabilizar el programa sin crashear.

Árboles

El programa del árbol se hizo similar al de los grafos y dígrafos, excepto que este debía tener una jerarquía de padres e hijos con el fin de que se fueran creando las ramas y hojas de los árboles. Para esto se fue creando que para las conexiones se fueran teniendo el padre, el hijo, la arista y el tiempo además del costo.

Al término de esta práctica pudimos tener el programa de árboles que pudiera ir creando los padres e hijos para después plasmarlos en el Json además de que pudiera el programa leer las conexiones y los nodos para poder tener las ramas y terminar el programa del árbol.

Errores

Durante la realización de esta práctica ocurrieron varios errores como que habían algunos errores en el guardado del archivo, ya que la jerarquía llegaba a provocar errores, además de algunos errores de compilación por algunos lados al olvidar colocar un punto y coma, lo cual se solucionaba rápido al ir revisando el código

